

소개

현장의 복잡한 업무를 사용자가 끊임 없이 처리할 수 있는 시스템으로 만드는 Java-Spring 백엔드 개발자입니다. 종이로 관리되던 학교 방문 점검을 일정 등록부터 점검표 작성, 서명, 재점검 이력까지 이어지는 시스템으로 바꾸고, 오래 걸리는 대량 첨부파일 작업은 비동기로 분리해 진행 상태를 확인한 뒤 결과를 내려받을 수 있도록 개선했습니다. 운영 담당자와 실제 업무 순서를 맞춘 뒤 화면 · 서버 · DB를 함께 설계하며, 테스트와 배포 이후의 운영까지 책임져 왔습니다.

기술 스택

Backend Java, Spring Boot, Spring MVC, Spring Security, MyBatis**Database** Oracle, MariaDB, MySQL**Frontend** **실무** JavaScript, Vue, HTML, CSS, JSP, WebSquare5, jQuery**프로젝트** TypeScript, React, Next.js, Redux Toolkit, Pinia**Infra · 배포** Linux, Tomcat, Jenkins, Gradle, Maven, Git, SVN

경력

유한책임회사 티지나래 웹 개발자 · 솔루션사업팀

2024.06 ~ 현재 · 정규직

B2B 협력사 포털 (PPS)

2025.02 ~ 현재

본사와 약 500개 협력사가 파트너 등록 · 계약, 824명의 현장 엔지니어(CE) 교육 · 자격 · 증빙 · 평가를 처리하는 B2B 업무 포털입니다. 운영 · 사업 담당자와 요구사항을 조율하고 기존 데이터 흐름을 분석해 DB 모델 · HTTP API · SQL 설계와 개발, 외부 시스템 연동, 사용자 검수, Jenkins 배포 · 운영까지 담당합니다.

사용 기술 | Java 21 · Spring Boot · MyBatis · MariaDB · Oracle · Vue

PPS 주요 성과

대량 첨부파일 다운로드 비동기 전환

- 문제** CE 300~400건의 첨부파일을 한 요청에서 수집 · 압축해 화면이 장시간 대기했고, 진행 중인지 실패한 것인지 확인할 수 없었습니다.
- 판단** 압축 속도보다 요청과 작업이 한 흐름에 묶인 구조가 문제라고 보고, 작업을 분리해 상태를 조회하는 방식을 선택했습니다.
- 구현** Spring 비동기 작업으로 압축을 분리하고, 요청 API는 작업 ID를 반환하며 상태 조회 API에서 진행 · 완료 · 실패를 확인하도록 구성했습니다. 작업 상태는 Hazelcast 분산 맵에 저장해 서버 2대 어디로 조회해도 같은 진행 상황을 보여주고, 새로고침 후에도 같은 작업을 이어서 확인할 수 있게 했습니다.
- 결과** 300~400건 단위 압축 작업이 HTTP 요청을 장시간 점유하던 문제를 해소하고, 사용자가 진행 상태를 확인하며 내려받도록 바꿨습니다.

분리돼 있던 본사 계정 발급 절차 통합

- 문제** 사내 계정 시스템과 포털에 계정을 각각 만들고 포털 ID는 DB에 직접 입력해야 해 누락과 정보 불일치 위험이 있었습니다.
- 판단** 두 시스템에 따로 입력하는 절차로는 누락을 막기 어렵다고 보고, 포털을 단일 발급 창구로 정했습니다.
- 구현** ID 생성, 조직정보 검증, 초기 인증정보 발급과 사내 시스템 연동을 한 절차로 묶고, 연동 실패 시 포털 저장도 취소되도록 트랜잭션을 구성했습니다.
- 결과** 사내 계정 생성 · 포털 계정 생성 · DB 입력의 3단계 절차를 단일 화면으로 통합했습니다. 계정 변경 전 재고 · 서비스 보유 여부도 확인해 잘못된 비활성화를 차단했습니다.

PPS 주요 성과 · 계속

서버 2대 환경의 비밀번호 초기화 인증번호 요청 제한

- 문제** 비밀번호 초기화가 인증번호를 알림톡으로 발송하는데 반복 클릭과 자동화된 요청을 막는 장치가 없었고, 서버가 2대라 서버별 메모리 제한만으로는 요청이 분산되면 제한이 무력화됩니다.
- 판단** 제한 상태를 서버 간에 공유해야 하고, 계정 · 전화번호 같은 식별값을 제한 키로 원문 저장하지 않아야 한다고 판단했습니다.
- 구현** Hazelcast 분산 맵에 TTL 쿨다운을 저장하고 putIfAbsent 기반 원자적 획득으로 동시 요청 중 한 건만 발송되게 했습니다. 제한 키는 계정 · 전화번호의 SHA-256 해시를 쓰고, Hazelcast가 없는 환경은 로컬 맵으로 폴백하며, 인증번호는 SecureRandom으로 생성했습니다.
- 결과** 어느 서버로 요청해도 같은 제한이 적용되고 화면에는 남은 대기 시간이 안내됩니다. 만료 직후 획득과 동시 요청 경쟁 같은 경계 조건은 관련 단위 테스트 30건으로 고정했습니다.

알림톡 발송 정책 공통화

- 문제** 기능마다 수신자와 발송 조건이 소스에 따로 작성돼 정책 변경 시 여러 코드를 수정해야 했습니다.
- 판단** 정책이 바뀔 때마다 코드를 배포하지 않도록, 자주 변하는 기준과 공통 발송 로직을 분리했습니다.
- 구현** 수신자와 발송 조건을 DB에서 관리하고, 공통 발송 서비스가 유형별 정책을 조회하도록 구현했습니다. 주요 조건과 예외 경로는 단위 테스트로 고정했습니다.
- 결과** 4개 업무 서비스의 5개 발송 지점을 공통 서비스 1곳으로 모았고, DB 설정 변경과 단위 테스트 4건으로 정책 변경 범위를 검증할 수 있게 했습니다.

교육용 단말 운영 시스템 (TSMS)

2025.09 ~ 현재

서울시교육청 디벗 사업에서 약 11만 대 교육용 단말의 등록부터 배송 · 설치, A/S, 점검까지 이어지는 업무와 이력을 관리합니다. 운영 담당자와 업무 규칙 · 기준 데이터를 조율하고 DB 모델 · HTTP API · SQL 설계와 화면 · 서버 개발, 외부 연동, 사용자 검수와 배포를 담당하고 있습니다.

사용 기술 | WebSquare5 · JavaScript · JSP · Java 11 · Spring MVC · MyBatis · MariaDB

TSMS 주요 성과

외부 API 키 노출 제거 및 호출 경로 공통화

- 문제** 25개 화면에서 지도 · 주소 및 교육행정정보 API를 직접 호출해 키가 브라우저에 노출되고, 연동 정보가 바뀔 때마다 각 화면을 수정해야 했습니다.
- 판단** 키만 교체하면 노출과 반복 수정이 재발한다고 보고, 외부 호출 책임을 화면에서 서버로 옮겼습니다.
- 구현** 외부 API 호출을 서버 공통 API 계층으로 모으고 키와 연동 URL을 서버 설정으로 분리했습니다. 관련 화면은 동일한 경로와 응답 형식을 사용하도록 전환했습니다.
- 결과** 25개 화면의 브라우저 키 노출을 제거하고, 화면마다 흩어져 있던 호출과 설정 변경 지점을 서버 공통 경로 1곳으로 모았습니다.

운영 중 점검 데이터의 시리얼 기준 재정립과 중복 정리

- 문제** 점검 대상이 입력용 14자리 시리얼로 저장돼 15자리 전체 시리얼과 혼재했고, 같은 시리얼 후보가 여러 건이면 잘못 매핑되거나 중복 저장될 수 있었습니다. 이미 운영 중이라 화면만 고쳐서는 쌓인 데이터가 정리되지 않았습니다.
- 판단** 저장 기준을 15자리 전체 시리얼로 재정립하고, 기존 데이터는 점검 결과를 보존한 채 정리하되 운영 반영 전에 되돌릴 수 있는 절차를 함께 준비했습니다.
- 구현** 15자리 기준 저장으로 변경하고 14자리 입력은 후보 검색 뒤 학교 배정 정보로 확정하게 했습니다. 학년 · 반 · 번호와 점검 이력 우선순위로 중복을 정리하는 SQL을 작성하고, 백업 · 롤백 · 사후 검증 SQL을 함께 준비해 운영 DB에 반영했습니다.
- 결과** 점검 결과와 상세 정보를 보존한 채 중복 행을 제거했고, ID 기준 수정으로 같은 문제의 재발을 막았습니다. (2026.07 운영 반영)

TSMS 주요 성과 · 계속

대량 단말 등록 검증과 QR 발급

문제	엑셀로 단말을 대량 등록할 때 생산입고 내역에 없거나 이미 등록된 일련번호가 함께 들어올 수 있었습니다.
판단	등록 뒤 오류를 정리하기보다 저장 전에 기존 데이터와 중복을 확인하도록 검증 순서를 앞당겼습니다.
구현	입력 일련번호를 생산입고 정보와 대조하고, 기등록 단말은 기존 학교 · 사용자 정보를 반환했습니다. QR에는 내부 식별자 대신 암호화 토큰을 사용했습니다.
결과	총 111,593대 중 108,237대의 QR 발급 흐름에 적용해 잘못된 단말 등록과 내부 식별자 노출을 사전에 차단했습니다. (2026.07 기준)

60초에도 못 끝나던 통합 뷰 조회의 성능 개선

문제	AS 이력 · 정산 세부현황 조회가 여러 테이블을 합쳐 둔 통합 뷰를 거치는데, 데이터가 쌓이면서 화면이 수십 초를 기다리는 수준까지 느려졌습니다.
판단	실행계획을 확인하니 고객 조건이 뷰 안까지 전달되지 못해 뷰 전체를 만들어 놓고 거르는 구조였고(추정 약 77억 행), 인덱스 추가가 아니라 조회 구조 자체를 바꿔야 한다고 판단했습니다.
구현	통합 뷰를 걷어내고 화면이 실제로 쓰는 컬럼만 기본 테이블 조인으로 재작성해, 고객 조건이 처음부터 인덱스로 적용되게 했습니다.
결과	운영 DB 재측정(2026.09, 이력 최다 고객 기준)에서 기존 쿼리는 60초 상한 내 완료 불가, 개선 쿼리는 77~124ms로 같은 결과를 반환했습니다 — 최소 770배 이상 단축. 실행계획 추정 행 약 77억 → 1,852행.

개인 프로젝트

ticket-rush · 선착순 티켓팅 예매 시스템

2026.08 ~ 진행 중 · 개인

동시 요청이 몰려도 이중 예매 0건을 목표로 하는 예매 시스템입니다. Redis SET NX 선점, 만료 홀드의 결제 거부, DB (회차 · 좌석) 유니크 제약의 세 겹 방어를 두고, Redis 장애 상황을 통합 테스트로 재현해 정확히 1건만 남는 것을 검증했습니다. 역등 처리 · 트랜잭셔널 아웃박스 · SSE 좌석 상태 스트림을 구현했고, 1석 100동시요청 경합 테스트에서 성공 1건 · 중복 예매 0건을 확인했습니다. 자동화 테스트 65건.

사용 기술 | Java 21 · Spring Boot 4 · Spring Modulith · MySQL · Redis · Flyway · Testcontainers · GitHub Actions

ReachRich · 개인 투자 연구·운영 플랫폼

2026.03 ~ 현재 · 개인

토스증권 Open API 기반으로 계좌 추적 · KRX 데이터 수집 · 전략 검증 · 모의운용 · 대시보드를 6개 모듈로 재설계했습니다. 날짜 기준 역등 저장과 부분 실패 격리, FastAPI 조회 API, React 화면, GitHub Actions · 텔레그램 경보를 구현했으며 데이터 수집 · 전략 검증 · 운용 로직의 백엔드 자동화 테스트 205개를 통과했습니다.

사용 기술 | Python · FastAPI · SQLAlchemy · SQLite · Parquet · React · TypeScript · GitHub Actions

교육 및 학력

교육	삼성청년SW아카데미(SSAFY) 8기 · 웹 개발 과정 수료	2022.07 ~ 2023.06
학력	아주대학교 e-비즈니스학과 · 편입 · 학사 졸업	2018.03 ~ 2020.08
학력	California State University, Chico · Business Administration (E-Business) · 편입 전 재학	2014.01 ~ 2015.05
자격	SQLD(SQL 개발자) · 한국데이터산업진흥원	2024.09 취득